CUDA Sum Reduction

MGP 2025 Spring

Due: 25 May (SUN) 23:59

Hello, MGP class!
Happy CUDA, isn't it?

You will implement parallel sum reduction with GPU Programming. In summary, you need to run all seven versions of reduction kernels provided in lecture notes by writing the correct kernel invocation.

Your final code submission should execute the fastest version you have for 16777216 (2^{24}) item reduction.

[Overview]
Parallel reduction refers to algorithms which combine an array of elements producing a single value as a result.

Structure of template code:

```
/HW5
├── bin              # Executable files generated by the Makefile
├── cuobj            # CUDA object files produced during compilation
├── obj              # Main object files reside here
│   └── driver.o     # Contains main(); the only provided object file
├── Makefile         # Contains targets for compiling, cleaning, formatting, and submitting the project
└── src              # Source code directory
    ├── reduction_kernel.cu  # CUDA kernel source file for reduction operations
    └── reduction.h          # Header file for reduction functionalities
4 directories, 4 files
```

The following are variable descriptions. Please read carefully and fit your code for correct grading.

g_idata

- 1D host integer input sequence. The total summation value is not over 2^31-1 or not under -2^31

g_odata

- 1D host integer output sequence. This is for copying the cuda result to host. This process will work in the main function so you don't need to care about it.

d_idata

- 1D device integer input sequence. This is for copying the sequence to the device. This process will work in the main function so you can assume that the sequence is already stored in d_idata.

d_odata

- 1D device integer output sequence. Your result must be stored in d_odata[0]

n

- the size of input sequence.

You don't include the time for memcpy and allocating the device memories. In case you want to do something about the allocation or copy (You probably don't) we have put separate alloc/dealloc/memcpy functions. In the driver.cc that is provided as pre-compiled binaries, it would look something like this:

```
int main() {
        ...
        g_idata = new int[N];
        g_odata = new int[N];
        allocateDeviceMemory(&d_idata);
        allocateDeviceMemory(&d_odata);
```

```
        cudaMemcpyToDevice(d_idata, g_idata, N*sizeof(int));
        time_start();
        reduction_optimized(g_idata, g_odata, d_idata, d_odata, N);
        time_end();
        cudaMemcpyToHost(g_odata, d_odata, sizeof(int));
        deallocateDeviceMemory(&d_idata);
        deallocateDeviceMemory(&d_odata);
        free g_idata, g_odata;
        …
}
```

If you want to allocate anything else, you should do it within reduction_optimized(). Of course, you're not allowed to change or re-write the main().

You can run reduction with the "make" command. Try: "make 1", "make 2", "make 3" and "make 4"

[Code - 10 pts]

- 10 pts if your program works and its runtime with size 16777216 is below 1.2 msec. No partial points.

The submission should still include **all 7 versions of code, including the kernel invocation** for 7 different versions.

[Report - 0 pt]

No report. Just enjoy Akaraka! Please send a photo from Akaraka via TA's email. The best photo will be featured as the wallpaper for the next homework announcement.

[Note]

Your code's speed is measured based on the maximum of 5 executions on the grading server. I won't regrade due to "It worked on my machine" excuses, so I encourage you to perform the assignment more generously than the criteria. As many students tend to rush near the deadline, your code may appear slower in the local environment. Therefore, I

recommend completing the assignment as early as possible when the server is less congested.

[Rules]

1. Put your HW5 directory under <your_home>/mgp/hw5
    a. All your make commands, especially 'make run' should work.
2. Do not change any of the output(i.e. 'std::cout') statements. We will grade based on the program outputs. Any change on the outputs would probably cause an error, leading to 0 points for your HW assignment.
3. Don't touch anything in the directory after the deadline. If any of the timestamps are later than the deadline, we will treat it as 'late submission'.
4. You can't use CUDA libraries for this assignment.
5. Please retain adequate permission on your files to avoid plagiarism.
6. No abusing. Please use your common sense.
7. Zero GPT policy. No mercy.